



Today's agenda

↳ Morris traversal

↳ Height of tree

↳ Diameter of tree



AlgoPrep



LNR

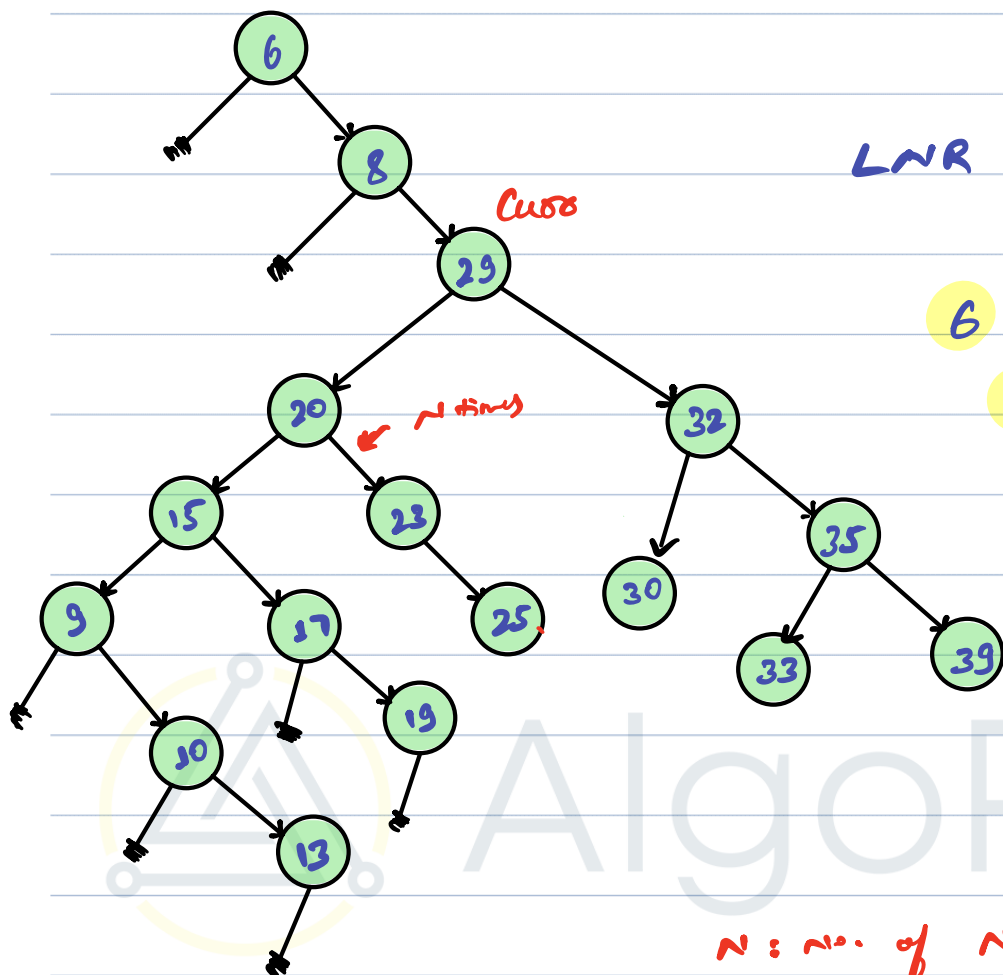
→ recursion → T.C: $O(N)$
S.C: $O(H)$

Morris Inorder traversal

Morris:

↓
T.C: $O(N)$ → iteration
S.C: $O(1)$

LNR



6 8 9 10 13 15 17
19 20

N : no. of Nodes

$E = N - 1$



//PSuedo code

```
void inorder(Node root){
```

```
    Node curr = root;
```

```
    while (curr != null) {
```

```
        if (curr.left == null) {
```

```
            s.o.p(curr.val);
```

```
            curr = curr.right;
```

```
        }
```

```
        else {
```

```
            Node currPl = curr.left;
```

```
            while (currPl.right != null && currPl.right != curr) {
```

```
                currPl = currPl.right;
```

```
            }
```

```
            if (currPl.right == null) {
```

```
                currPl.right = curr;
```

```
                curr = curr.left;
```

```
            }
```

```
            else { // currPl.right == curr
```

```
                currPl.right = null;
```

```
                s.o.p(curr.val);
```

```
                curr = curr.right;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

T.C: $3 \times n \log(n)$

S.C: $O(1)$

6 8 9 10 13 15 17 19 20 23
25 29 32 35 39



```
void Inorder(Node root){
```

```
Node Curr = root;
```

LNR

```
while (Curr != null) {
```

```
if (Curr.left == null) {
```

```
    s.o.p(Curr.val);
```

```
    Curr = Curr.right;
```

```
}
```

```
else {
```

```
    Node CurrPl = Curr.left;
```

```
while (CurrPl.right != null && CurrPl.right != Curr) {
```

```
    CurrPl = CurrPl.right;
```

```
}
```

```
if (CurrPl.right == null) {
```

```
    CurrPl.right = Curr;
```

```
    Curr = Curr.left;
```

```
}
```

```
else if (CurrPl.right == Curr)
```

```
    CurrPl.right = null;
```

```
    s.o.p(Curr.val);
```

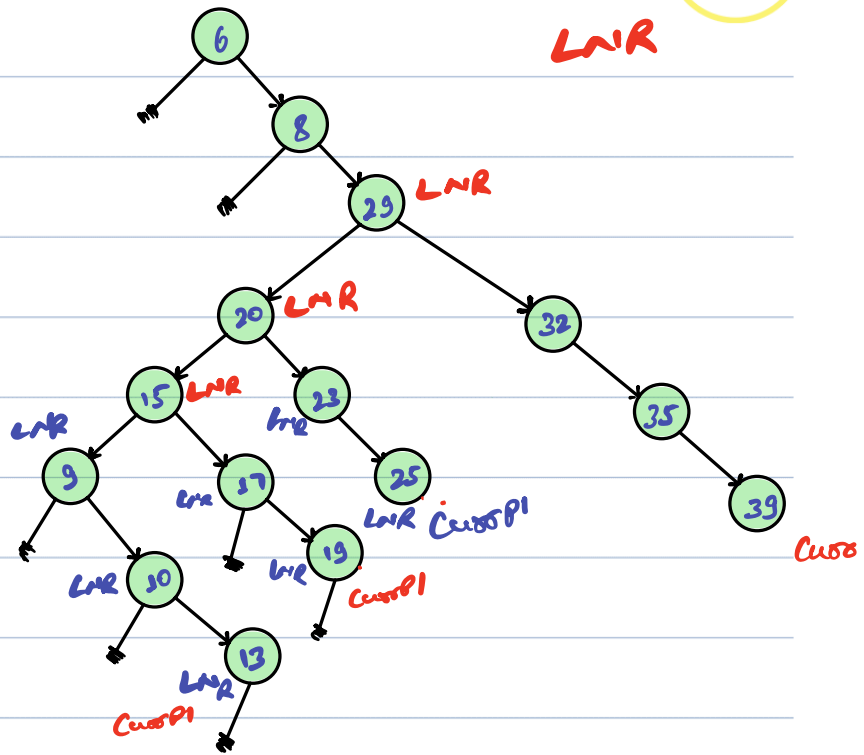
```
    Curr = Curr.right;
```

```
}
```

```
}
```

```
}
```

```
}
```



AlgoPrep

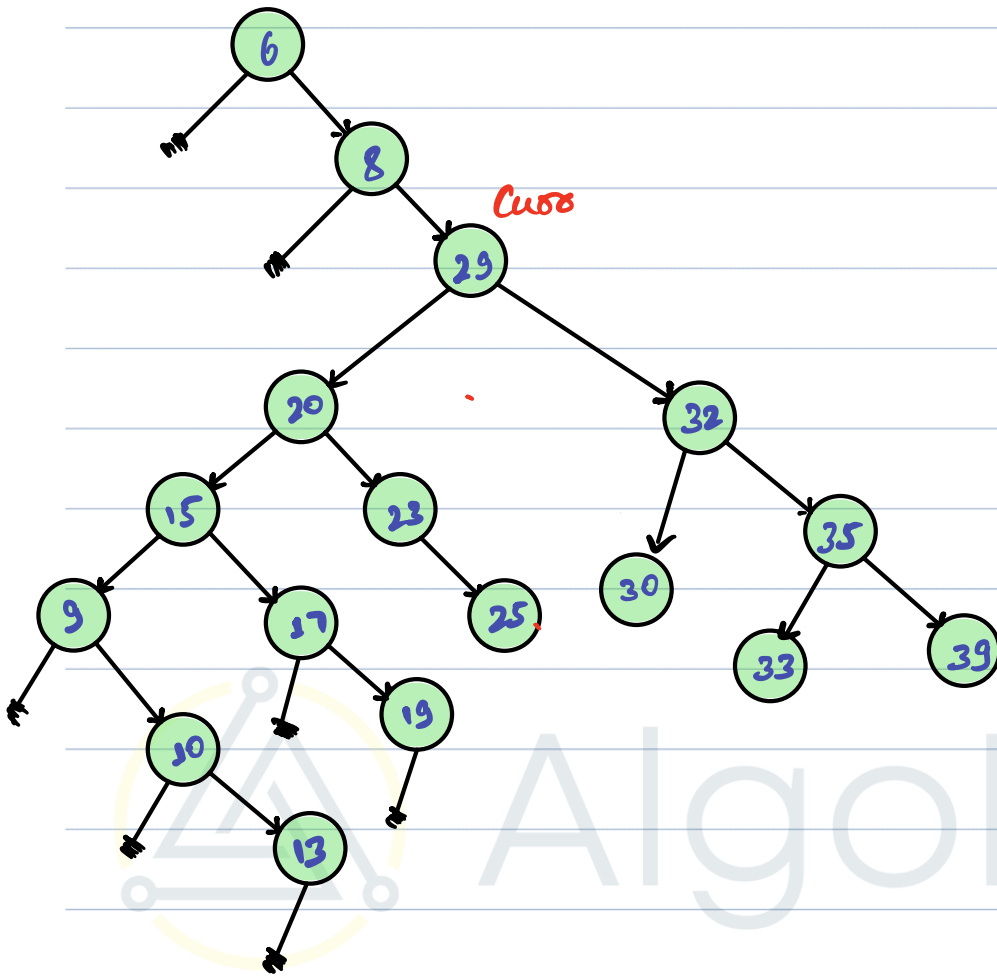


→ ~ LR

morris

Preorder

Traversal





//Pseudo code

```
void Inorder (Node root) {
```

```
    Node Curr = root;
```

```
    while (Curr != null) {
```

```
        if (Curr.left == null) {
```

```
            s.o.p (Curr.val);
```

```
            Curr = Curr.right;
```

```
        }
```

```
        else {
```

```
            Node CurrPl = Curr.left;
```

```
            while (CurrPl.right != null && CurrPl.right != Curr) {
```

```
                CurrPl = CurrPl.right;
```

```
            }
```

```
            if (CurrPl.right == null) {
```

```
                CurrPl.right = Curr;
```

```
                s.o.p (Curr.val);
```

```
                Curr = Curr.left;
```

```
            }
```

```
            else { // CurrPl.right == Curr
```

```
                CurrPl.right = null;
```

```
                s.o.p (Curr.val);
```

```
                Curr = Curr.right;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

T.C: $3 \times N \times O(N)$

S.C: $O(1)$

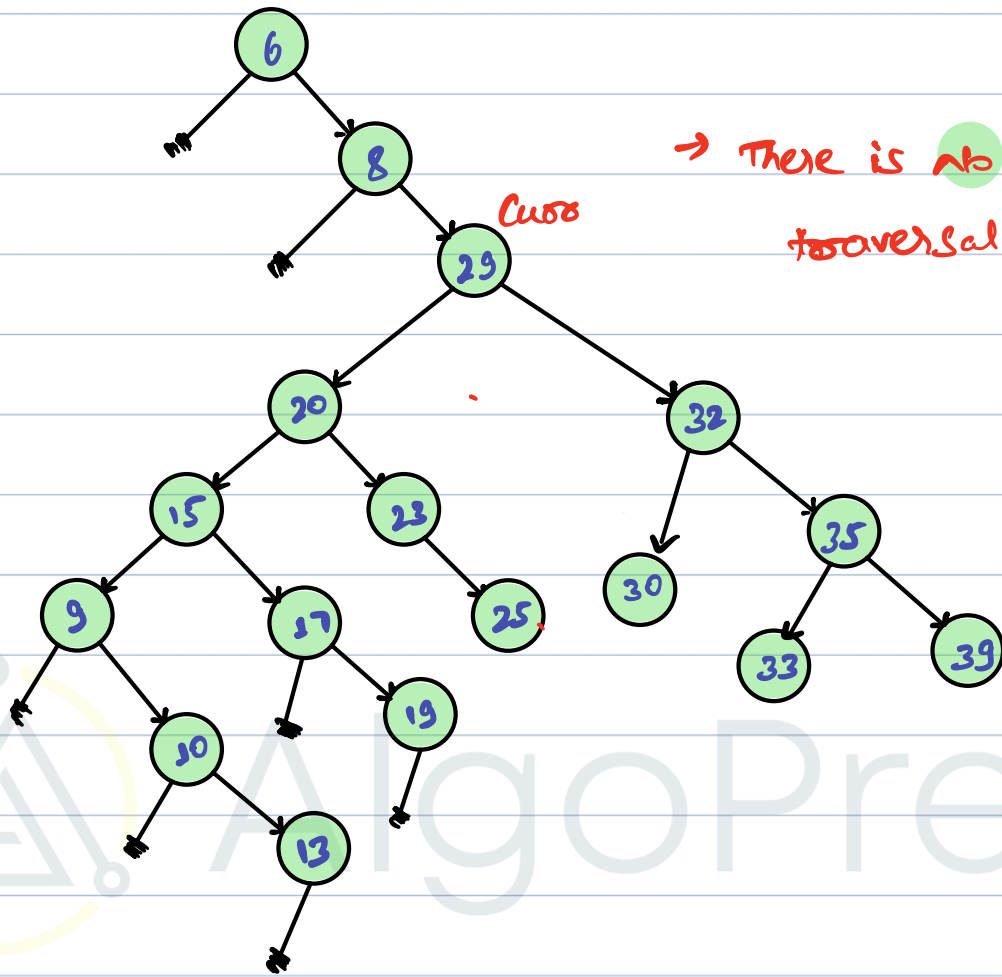


→ LRN

morris

Postorder

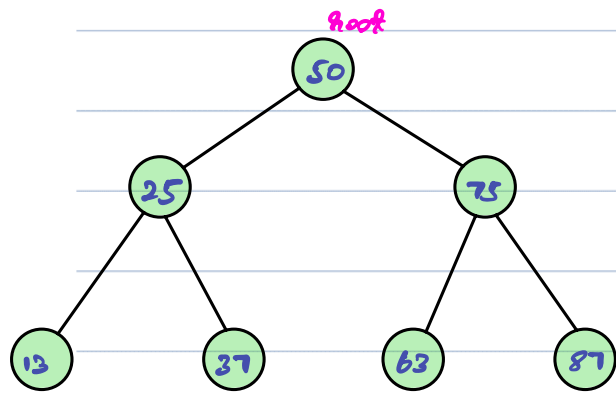
traversal



→ There is no morris traversal for Postorder.

AlgoPrep

NRL

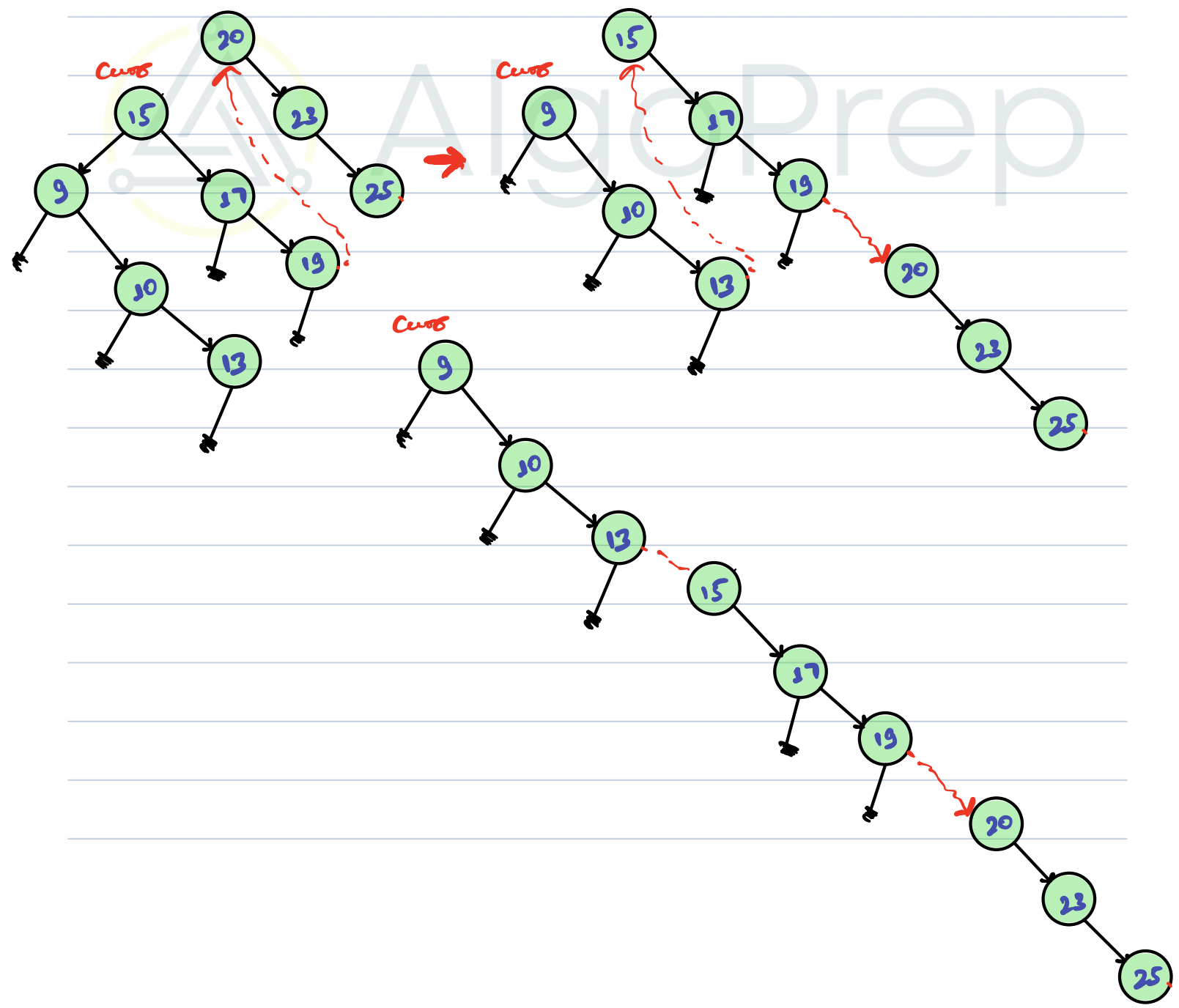


→ Postorder: 13 37 25 63 87 75 50

→ Reverse Preorder: 50 75 87 63 25 37 13

↓
13 37 25 63 87 75 50

Insight on Morris traversal





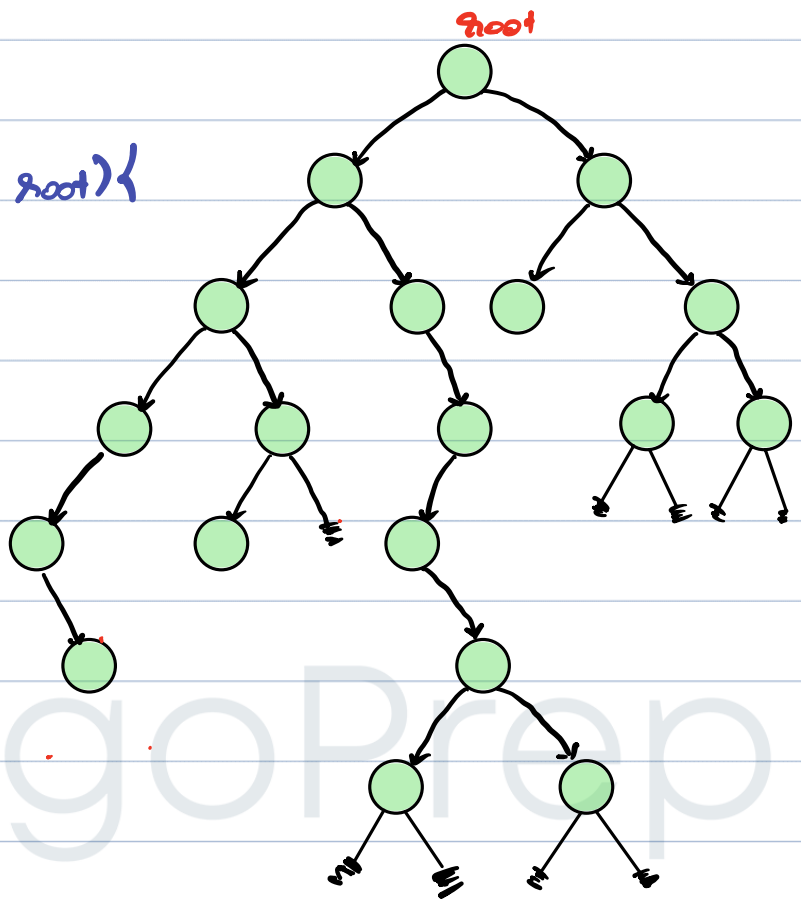
Q) Given root Calculate longest Path across root of the tree.
↳ in terms of edge

```
int maxPathAcrossRoot(Node root){
```

```
    int lht = height(root.left);  
    int rht = height(root.right);
```

```
    return lht + rht + 2;
```

```
}
```



T.C: $O(N)$

S.C: $O(H)$



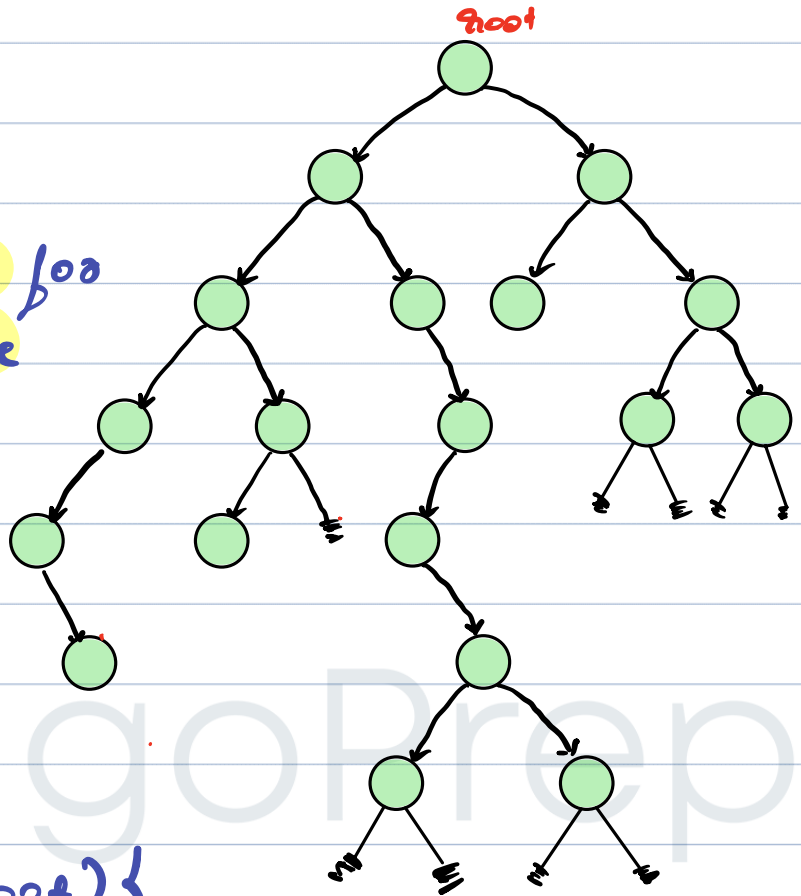
Q) Given root Calculate longest Path across an
 ↳ in terms of edge

node of the tree.
 ↳ Diameter of the tree

// idea 1

↳ Call `maxPathAcrossRoot` for every node and Pick the maximum.

T.C: $O(N^2)$



// idea 2

int ans = 0;

int height(Node root) {
 if (root == null) { return -1; }

T.C: $O(N)$

S.C: $O(N)$

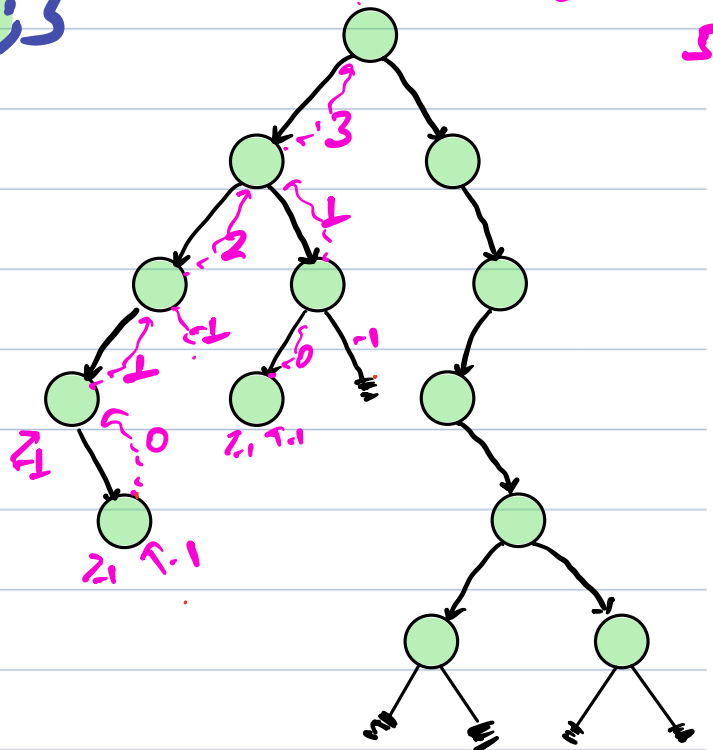
int lht = height(root.left);
 int rht = height(root.right);

ans = max(ans, lht + rht + 2);

return max(lht, rht) + 1;

}

ans = 0 0 1 2 3 4 5



int main() {
 height(root);
 return ans;

}